

Attention I: Tokens Talking to Each Other

How words borrow meaning from their neighbours · video 2:22

This lesson has one big idea: **attention lets every token look at the other tokens and borrow meaning from the ones that matter**. Each token asks a question (its query), compares it with what every token offers (its key), and mixes in the information (the values) of the best matches.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Context changes meaning	▶ 0:08-0:43	3	☐
2. Attention weights: a budget of 1	▶ 0:43-0:55	4	☐
3. Query, key and value	▶ 0:55-1:21	3	☐
4. Matching with the dot product	▶ 1:21-1:33	4	☐
5. Mixing in the values	▶ 1:33-1:44	4	☐
6. Only looking backwards	▶ 1:44-1:55	4	☐
7. Learned, not hand-written	▶ 1:55-2:20	4	☐

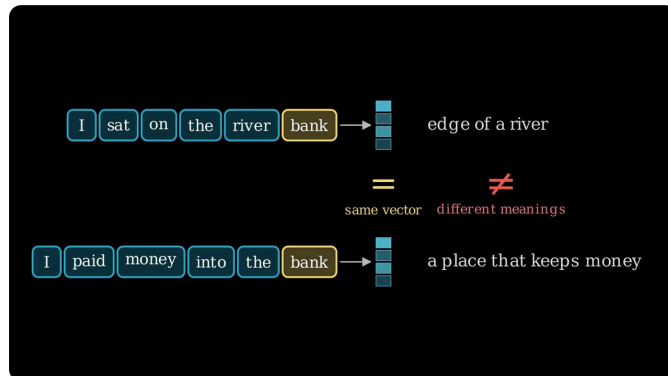
Before you start: this lesson builds on episodes 3 and 4: every token enters attention as one vector that says what it is and where it is. Concept 4 uses the dot product (Foundations F02) and concept 7 multiplies a vector by a matrix (F03); F07 (Softmax, Properly) prepares episode 6, where scores become weights. The code exercise in concept 5 uses NumPy (F11).

CONCEPT 1 OF 7

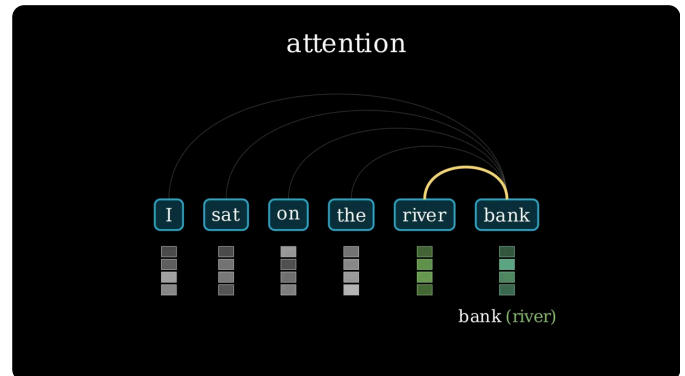
Context changes meaning

▶ Video 0:08–0:43

Each token's vector says **what it is** and **where it is**, but it was made **on its own**, without looking at any other word. Take *bank*: in “*I sat on the river bank*” it is the edge of a river; in “*I paid money into the bank*” it is a place that keeps money. Same token, same starting vector, two meanings.



▶ 0:30 Same token, same vector, two meanings.



▶ 0:42 bank notices river and updates its vector.

KEY IDEA

Attention lets every token **look at the other tokens** and pull in information from the ones that matter. *bank* notices *river* and updates its own vector to mean “riverbank”: context changes meaning.

Check 1: Context changes meaning

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

In both sentences, *bank* is the 6th token. Before attention, are its two vectors the same or different? Why?

.....

1.2 MULTIPLE CHOICE

What does attention let a token do?

- A. Swap itself for a different, less ambiguous token
- B. Look at the other tokens and pull in information from the ones that matter
- C. Look the word up in a built-in dictionary of meanings
- D. Delete the words that don't matter from the text

1.3 SHORT ANSWER

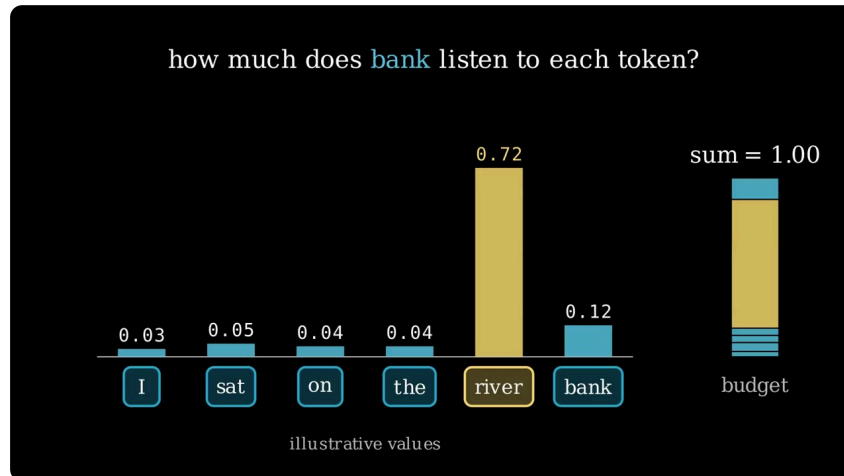
In “*The baseball player swung the bat*”, which earlier word should *bat* pay most attention to, and what should its updated vector mean?

.....
.....

Attention weights: a budget of 1

▶ Video 0:43–0:55

Each token decides **how much to listen** to every other token. These numbers are the **attention weights**. They work like a **budget**: every weight is between 0 and 1, and together they **add up to 1**. Here *bank* spends 0.72 of its attention on *river*, and only 0.04 each on *the* and *on*. Notice that a token also gives some weight to **itself** (0.12).



▶ 0:54 How much bank listens to each token: 0.72 on river, very little on “the” and “on”. The weights add up to 1, like a budget.

KEY IDEA

For each token, the attention weights are all ≥ 0 and **add up to 1**. A large weight means “listen closely to this token”; a tiny one means “mostly ignore it”.

Check 2: Attention weights: a budget of 1

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

From the chart, how much of *bank*'s attention goes to tokens other than *river*?

2.2 MULTIPLE CHOICE

Which list could be one token's attention weights over four tokens?

- A. 0.5, 0.5, 0.5, 0.5
- B. 0.1, 0.2, 0.3, 0.4
- C. 0.9, 0.2, -0.1, 0.0
- D. 0.25, 0.25, 0.25, 0.2

2.3 TRUE OR FALSE

“If *bank* starts paying more attention to *river*, it must pay less attention to at least one other token.”

True ☐ False ☐

2.4 CALCULATION

In “*I paid money into the bank*”, suppose *bank* gives 0.60 to *money*, 0.12 to itself, and splits the rest equally over *I*, *paid*, *into* and *the*. What weight does each of those four get?

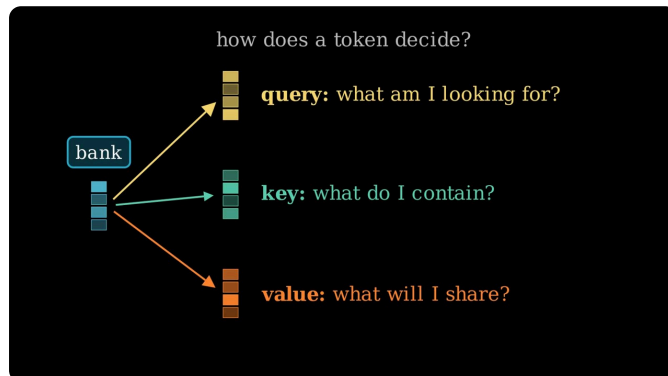
CONCEPT 3 OF 7

Query, key and value

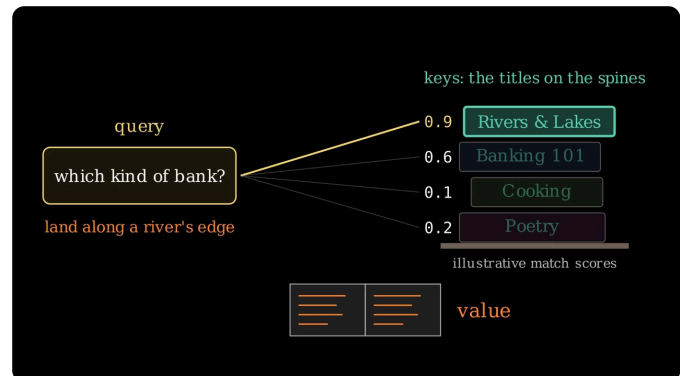
▶ Video 0:55–1:21

How does a token decide whom to listen to? Every token produces **three new vectors**: a **query** (“what am I looking for?”), a **key** (“what do I contain?”) and a **value** (“what will I share, if someone listens to me?”).

Think of a library. Your **query** is the question you bring (*which kind of bank?*). Each book has a **key**, like the title on its spine. You compare your question with every title, and the better the match, the more you read from that book's contents: its **value**.



▶ 1:06 Every token makes a query, a key and a value.



▶ 1:21 The library: read most from the best-matching title.

KEY IDEA

The **query** and the **key** are compared to decide **how much** a token listens. The **value** is **what gets passed along** to the token that listens.

Check 3: Query, key and value

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

In the library analogy, what plays the role of a token's **key**?

- A. The question you bring
- B. The title on a book's spine
- C. The text inside the book
- D. The shelf the book stands on

3.2 MULTIPLE CHOICE

To decide how much *bank* listens to *river*, which two vectors are compared?

- A. bank's query and river's key
- B. bank's key and river's query
- C. bank's value and river's value
- D. bank's query and river's value

3.3 SHORT ANSWER

The library's match scores were *Rivers & Lakes* 0.9, *Banking 101* 0.6, *Cooking* 0.1, *Poetry* 0.2. Which book do you read most from, and which of the three vectors is the text you read?

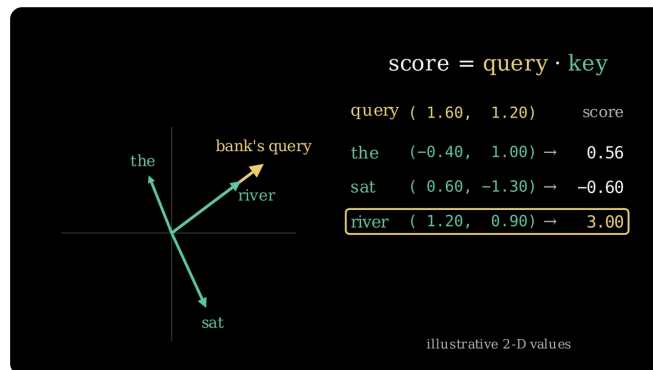
CONCEPT 4 OF 7

Matching with the dot product

▶ Video 1:21-1:33

The match between a query and a key is measured with the **dot product** from the embeddings episode (Foundations F02): multiply matching numbers, then add everything up. So **score = query · key**.

If *bank*'s query points in the **same direction** as *river*'s key, the score is high, and bank pays attention to river. In the video, river's key turns from pointing away (score -1.18) to lining up with the query (3.00), beating *the* (0.56) and *sat* (-0.60).



▶ 1:33 river's key points the same way as bank's query: top score.

KEY IDEA

score = query · key. Same direction → large score → more attention. At a right angle the score is 0; pointing away, it is negative.

Check 4: Matching with the dot product

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

bank's query is (1.6, 1.2) and *river*'s key is (1.2, 0.9). Compute the score.

4.2 CALCULATION

Two new keys: *water* (0.8, 0.6) and *money* (−0.6, 0.8). Compute each one's score with bank's query (1.6, 1.2). Which would bank listen to more?

4.3 MULTIPLE CHOICE

As river's key turns towards bank's query, its length stays 1.5. What happens to its score?

- A. It goes down
- B. It goes up, from -1.18 to 3.00
- C. It stays the same, because the length does not change
- D. It becomes exactly 1

4.4 TRUE OR FALSE

"*sat*'s score is -0.60 . A negative score means the angle between the query and the key is more than 90° ."

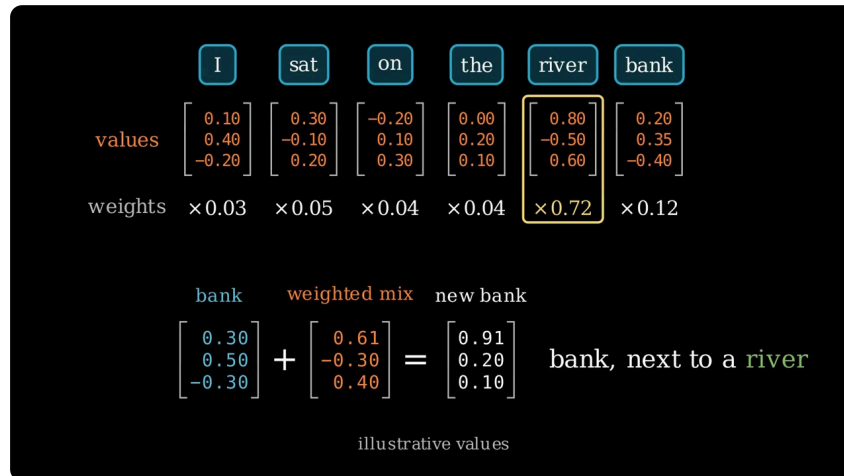
True ☐ False ☐

CONCEPT 5 OF 7

Mixing in the values

▶ Video 1:33–1:44

Once the weights are known, *bank* collects a **weighted mix of all the values**: multiply each token's value by its weight and add the results. River's weight is 0.72, so the mix is **mostly river's value**. Then bank **adds** the mix to its own vector; it does not replace it. Now its vector doesn't just mean *bank*: it means “bank, next to a river”.



▶ 1:44 Each value times its weight, added up: a weighted mix, mostly river's. The mix is added to bank's own vector.

KEY IDEA

new vector = old vector + the weighted mix of the values (weight $\times$ value, added up over all tokens). The weights decide how much of each value flows in.

Check 5: Mixing in the values

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

River's value is (0.80, -0.50, 0.60) and its weight is 0.72. What does river alone contribute to the mix? Round to two decimals.

5.2 CALCULATION

A token attends to just two tokens, with weights 0.25 and 0.75. Their values are (0.8, 0.2) and (0.0, 0.4). What is the weighted mix?

5.3 TRUE OR FALSE

“After attention, *bank*'s vector is replaced by the weighted mix.”

True ☐ False ☐

5.4 HANDS-ON CODE

Try it yourself. The code computes bank's update with NumPy, using the video's numbers. (a) What does it print? (b) In words, what does `w @ V` compute for each of the 3 columns? (c) Change `w` so that bank listens *only* to river (weight 1 on river, 0 elsewhere). What is the mix now, and why?

```
import numpy as np

# tokens:           I      sat    on    the    river bank
w = np.array([0.03, 0.05, 0.04, 0.04, 0.72, 0.12]) # bank's weights
V = np.array([[ 0.10,  0.40, -0.20],                # one value per token
               [ 0.30, -0.10,  0.20],
               [-0.20,  0.10,  0.30],
               [ 0.00,  0.20,  0.10],
               [ 0.80, -0.50,  0.60],
               [ 0.20,  0.35, -0.40]])
x_bank = np.array([0.30, 0.50, -0.30])              # bank's own vector

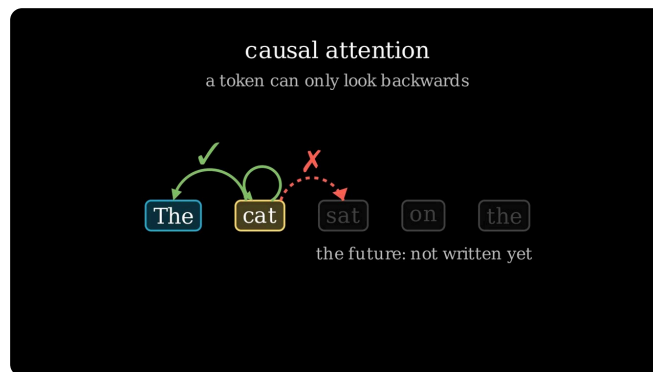
mix = w @ V
print(mix.round(2))
print((x_bank + mix).round(2))
```

CONCEPT 6 OF 7

Only looking backwards

▶ Video 1:44–1:55

One rule for language models: a token can only **look backwards**, at itself and the tokens before it. The model writes one word at a time (episode 1), so when it predicts the next word, the future isn't written yet. In “*The cat sat on the*”, *cat* can look at *The* (and at itself), but never at *sat*. This is called **causal attention**.



▶ 1:54 Causal attention: cat can look at The and at itself, never at sat.

KEY IDEA

In causal attention each token attends only to **itself and earlier tokens**. Later tokens get no attention at all.

Check 6: Only looking backwards

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 SHORT ANSWER

In “*The cat sat on the*”, list every token that *on* is allowed to look at.

.....

6.2 CALCULATION

In the 5-token text “*The cat sat on the*”, each token may look at some tokens. Count all allowed (looker, looked-at) pairs. How many of the 25 possible pairs are forbidden?

.....

.....

6.3 MULTIPLE CHOICE

Why can a token not look at the tokens after it?

- A. Looking forward would be too slow to compute
- B. When the model generates text, the later words have not been written yet
- C. Later words never help with meaning
- D. The dot product only works in one direction

6.4 TRUE OR FALSE

“In “*I sat on the river bank*”, *bank* is allowed to look at every token of the sentence.”

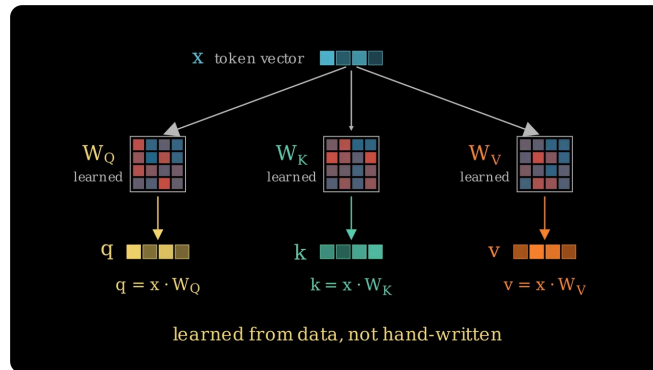
True ☐ False ☐

CONCEPT 7 OF 7

Learned, not hand-written

▶ Video 1:55-2:20

Each query, key and value is made by **multiplying the token's vector x by a matrix** (Foundations F03): $q = x \cdot W_Q$, $k = x \cdot W_K$, $v = x \cdot W_V$. The matrices are **learned in training**. Nobody tells the model that *river* explains *bank*; it figures that out from data.



▶ 2:09 q , k and v : the token's vector times three learned matrices.

KEY IDEA

q , k and v are the token's vector times **three learned matrices**: what to look for, what to advertise and what to share are all **learned from data**.

Check 7: Learned, not hand-written

Answer all 4 questions before looking at the answers. Pass mark: all correct.

7.1 MULTIPLE CHOICE

Where do W_Q , W_K and W_V come from?

- A.** Engineers write rules into them, like "river explains bank"
- B.** They are learned from data during training
- C.** They are copied from the embedding table
- D.** They are random and never change

7.2 CALCULATION

In the video's picture, the token vector x has 4 numbers and each matrix is a 4×4 grid. (a) How many numbers are in the query q ? (b) How many learned numbers are in W_Q , W_K and W_V together?

7.3 PUT IN ORDER

Put the steps of attention for *bank* in order: *add the mix to bank's vector* · *make q , k and v for every token* · *take the weighted mix of the values* · *compare bank's query with every key* · *turn the scores into weights that add up to 1*.

7.4 TRUE OR FALSE

"Every token is multiplied by the same W_Q , so *bank* and *river* get different queries only because their vectors x are different."

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Context changes meaning · rewatch ▶ 0:08-0:43

1.1 The same. The vector only encodes what the token is (*bank*) and where it is (position 6), and both are identical. *river* and *money* have not been looked at yet.

1.2 B. The token stays the same token (*bank* never becomes a new token); only its vector is updated with information from the other tokens.

1.3 *baseball* (or *swung*): a baseball bat, not the flying animal. Like *river* for *bank*, that word tells *bat* which meaning is meant.

2. Attention weights: a budget of 1 · rewatch ▶ 0:43-0:55

2.1 0.28. $1 - 0.72 = 0.28$ (check: $0.03 + 0.05 + 0.04 + 0.04 + 0.12 = 0.28$).

2.2 B. A adds up to 2; C adds up to 1 but has a negative weight; D adds up to only 0.95. B is all ≥ 0 and adds up to exactly 1.

2.3 True. The total is fixed at 1, like a budget: spending more in one place means spending less somewhere else.

2.4 0.07. $1 - 0.60 - 0.12 = 0.28$ is left, and $0.28 \div 4 = 0.07$.

3. Query, key and value · rewatch ▶ 0:55-1:21

3.1 B. A is the query and C is the value. The key is what your question is compared with.

3.2 A. The token that asks uses its query; the token being considered offers its key. Values do not decide the amount; they are only used afterwards, for the mix.

3.3 *Rivers & Lakes*; the value. The highest match gets the most reading, and a book's contents ("land along a river's edge") are its value.

4. Matching with the dot product · rewatch ▶ 1:21-1:33

4.1 3.00. $1.6 \times 1.2 + 1.2 \times 0.9 = 1.92 + 1.08 = 3.00$, the value in the video.

4.2 water 2.00, money 0.00; water. $1.28 + 0.72 = 2.00$ and $-0.96 + 0.96 = 0$. *water* points exactly the same way as the query (it is half of it); *money* is at a right angle to it.

4.3 B. The dot product depends on direction as well as length: the smaller the angle to the query, the bigger the score.

4.4 True. Less than 90° gives a positive score, exactly 90° gives 0, more than 90° (pointing away) gives a negative score.

5. Mixing in the values · rewatch ► 1:33-1:44

5.1 (0.58, -0.36, 0.43). $0.72 \times \text{each entry} = (0.576, -0.36, 0.432)$. That is most of the full mix $(0.61, -0.30, 0.40)$.

5.2 (0.20, 0.35). $0.25 \times (0.8, 0.2) + 0.75 \times (0.0, 0.4) = (0.20, 0.05) + (0.00, 0.30) = (0.20, 0.35)$. It lies closer to the value with the bigger weight.

5.3 False. The mix is **added** to bank's own vector: $(0.30, 0.50, -0.30) + (0.61, -0.30, 0.40) = (0.91, 0.20, 0.10)$. Bank keeps what it was and gains context.

5.4 (a) [0.61 -0.3 0.4] and [0.91 0.2 0.1] · (c) [0.8 -0.5 0.6] (a) The video's numbers (NumPy drops trailing zeros: -0.3 is -0.30). (b) For each column it multiplies the six values in that column by the six weights and adds them up, e.g. $0.03 \times 0.10 + 0.05 \times 0.30 + \dots + 0.12 \times 0.20 = 0.61$. (c) `w = np.array([0, 0, 0, 0, 1.0, 0])` gives exactly river's value: with the whole budget on one token, the mix is that token's value.

6. Only looking backwards · rewatch ► 1:44-1:55

6.1 The, cat, sat, on. Itself and everything before it. The final *the* comes later, so it is off limits.

6.2 15 allowed, 10 forbidden. The sees 1 token, cat 2, sat 3, on 4, the 5: $1 + 2 + 3 + 4 + 5 = 15$, and $25 - 15 = 10$.

6.3 B. Only the earlier words exist when each new word is predicted. C is false: later words can help, but they are not available yet.

6.4 True. *bank* is the last token, so every other token comes before it. That is why it can use *river*.

7. Learned, not hand-written · rewatch ► 1:55-2:20

7.1 B. They start random and training adjusts them; no rules are written by hand.

7.2 (a) 4. (b) 48. A 4-number vector times a 4×4 matrix gives 4 numbers; each matrix holds 16, and $3 \times 16 = 48$.

7.3 make q, k, v → compare query with every key → weights → weighted mix of values → add to bank's vector. Matching (query · key) decides the weights; the weights decide the mix; the mix is added.

7.4 True. One set of learned matrices serves every token; each token's own vector makes its query, key and value different.